

Accident System — Architecture Guide

This project detects road accidents from video using a multi-stage pipeline: find objects → track them → run 3 analysis phases → combine scores → alert.

There are two ways to run it, plus optional helper tools. Most logic lives in shared modules used by both runners.

Big picture

How you start the app

Step 1: See objects

Step 2: Three-phase analysis

Step 3: Final decision

Step 4: Output

app.py — Web dashboard

stream_processor.py — Live camera

detection/yolo_module.py

tracking/deepsort_module.py or vehicle_tracker.py

Phase A — proximity + TTC

Phase B — trajectory / stop / crash physics

Phase C — optical flow + bbox deformation

fusion/scoring.py

Optional XGBoost model

Optional CNN-LSTM model.py

templates/index.html

firebase_uploader.py

llm_vision_module.py

Two main entry points (pick one)

File What it is When to use

app.py

FastAPI web server with upload UI

Upload images/videos in browser, train XGBoost, view results

stream_processor.py

Live camera / RTSP pipeline

Webcam, traffic camera, or video file in real time

Run commands:

python app.py # <http://127.0.0.1:8000>

python stream_processor.py --source 0 # webcam

Shared brain — detection pipeline

These files do the actual accident detection logic. Both app.py and stream_processor.py use them (directly or through wrappers).

1. Configuration (single place for all thresholds)

File Role

config.py

All settings: proximity pixels, TTC frames, fusion weights, cooldown seconds, model paths, Firebase settings. Change thresholds here only.

2. Detection — “What is in the frame?”

File Role

detection/yolo_module.py

Loads YOLOv8 (yolov8n.pt). Detects person, car, bike, bus, truck in each frame. Used by the web app for static image detection.

3. Tracking — “Same vehicle across frames”

File Role

tracking/deepsort_module.py

Used by app.py. ByteTrack via YOLOv8. Assigns persistent IDs, stores position/speed history (30 frames). Exposes Track objects the phase modules read.

tracking/vehicle_tracker.py

Used by stream_processor.py. Same idea (ByteTrack) but with TrackedVehicle objects. Stream pipeline uses this tracker.

Both trackers solve the same problem; they exist because the web app and stream app were wired with slightly different class names.

4. Three-phase analysis — “Is this a crash?”

File Phase What it checks

phases/phase_a_proximity.py

Phase A

Are two objects close enough? Are they converging (Time-To-Collision)? Tighter threshold for vehicle-person pairs (80px vs 150px).

phases/phase_b_trajectory.py

Phase B

Did paths cross and one vehicle stop (IITH paper)? Emergency stop? Rear-end (speeds converged)? Spin, merge, occlusion.

phases/phase_c_anomaly.py

Phase C

Optical flow spike inside bbox? Sudden bbox shape change? Must persist 3 frames to reduce noise.

Helper math:

File Role

utils/geometry.py

Distance, line intersection, TTC, angle math

utils/optical_flow.py

Farneback optical flow, frame diff, flow dispersion

5. Decision — “Combine everything into one score”

File Role

fusion/scoring.py

Weighted fusion of signals. Current weights: trajectory stop 45%, TTC 20%, emergency stop 20%, optical flow 8%, dispersion 7%. CNN-LSTM weight = 0 until properly trained.

Optional ML layers on top:

File Role

model.py

ResNet18 + LSTM for sequence classification. Runs on video in app.py; result stored but not heavily weighted in fusion yet.

model_output/accident_xgboost.json

Trained XGBoost classifier. If present, app.py can use it instead of pure fusion.

accident_features.csv

Training data for XGBoost (logged from the web UI).

Web app only (app.py extras)

File Role

templates/index.html

Dashboard UI: upload image/video, sliders, results, train model button

static/uploads/

Processed images/videos saved here and served to browser

llm_vision_module.py

After video accident: tries Ollama LLaVA for text description; falls back to rule-based text from scores

app.py flow (video upload):

Read video frame by frame

ByteTrack (deepsort_module)

Phase A B C per frame

CNN-LSTM features (parallel)

fuse_scores() optional XGBoost

Require 3 consecutive frames + 20s cooldown before marking accident

Draw boxes, telemetry panel, write annotated MP4

Optional LLM description of worst frame

Live stream pipeline only

File Role

stream_processor.py

Main loop: camera track detect accident display / upload

accident_detector.py

Wraps phases A/B/C + fusion for stream. Optional Stage-1 YOLO (accident_model.pt) if file exists.

firebase_uploader.py

Uploads confirmed accidents to Firebase (or saves JSON locally if no Firebase key)

health_monitor.py

Sends Pi health (FPS, CPU temp, RAM) to Firebase every 30s

Tuning / development tools (optional)

File Role

data_logger.py

Run on video clips; writes CSV of speeds, distances, flow ratios per frame

threshold_analyzer.py

Reads that CSV; plots accident vs normal; suggests threshold updates for config.py

Documentation & data (not executed code)

File Role

README.md

Original project readme

tech_Des.md

Kishore's original technical design (before your PDF fixes)

project_updates.pdf

Analysis of problems + fixes (IITH stop, TTC, ByteTrack, etc.)

accident_features.csv

Labeled feature rows for XGBoost training

Folders you can ignore

Path Why

claude files/

Old copies of modules now moved to project root (config.py, stream_processor.py, etc.). Do not use these — use the root-level files instead.

pycache/

Python bytecode cache

venv/

Virtual environment (dependencies)

Mental model (simple)

Think of it in 4 layers:

Layer 1 — INPUT

app.py (browser upload) OR stream_processor.py (camera)

Layer 2 — PERCEPTION

YOLO detects objects ByteTrack follows them over time

Layer 3 — REASONING

Phase A: Are they close and heading toward each other?

Phase B: Did paths cross AND someone stop / crash physics?

Phase C: Is there chaotic motion / shape change?

Layer 4 — DECISION + OUTPUT

fusion/scoring.py combines signals

Web: annotated video + JSON response

Stream: Firebase alert + snapshot

Which file do I edit for...?

Goal Edit

Change proximity / TTC / cooldown / fusion weights

config.py

Fix false positives from trajectory logic

phases/phase_b_trajectory.py

Fix proximity / pedestrian detection

phases/phase_a_proximity.py + detection/yolo_module.py

Change final score formula

fusion/scoring.py

Change web UI

templates/index.html

Change upload API behavior

app.py

Live camera deployment

stream_processor.py + firebase_uploader.py

Tune thresholds from real videos

data_logger.py threshold_analyzer.py update config.py

Dependency chain (who imports whom)

config.py

detection/yolo_module.py tracking/deepsort_module.py app.py

utils/geometry.py phases/phase_a_proximity.py app.py,
accident_detector.py

utils/optical_flow.py phases/phase_c_anomaly.py

phases/phase_b_trajectory.py

fusion/scoring.py app.py, accident_detector.py

model.py app.py

tracking/vehicle_tracker.py stream_processor.py, data_logger.py

accident_detector.py stream_processor.py